

Cheap Sim2Real: teaching a \$150 arm a manipulation task with 10 real demonstrations

AMD AI DevMaster Hackathon — Track 3 (Physical AI) Genesis · LeRobot ACT · AMD Radeon GPU (ROCm) · SO-101

1. Target application

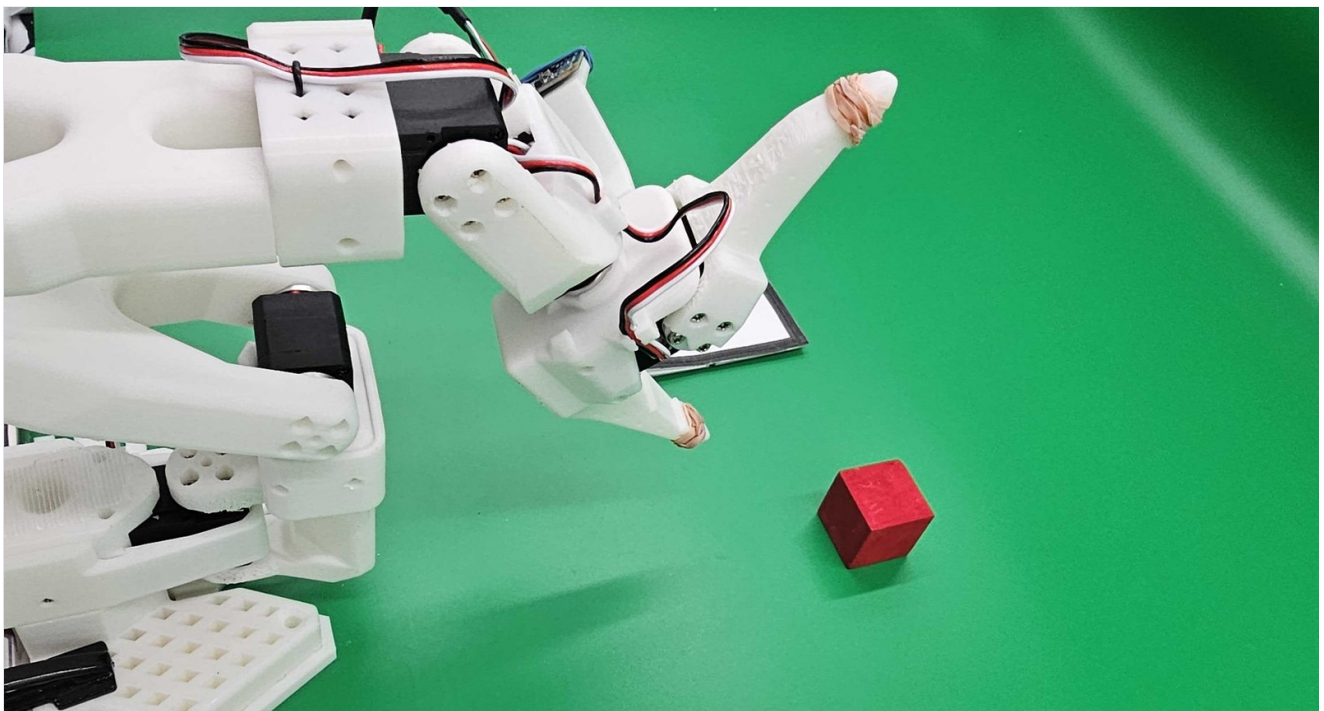
The problem. Imitation learning is the practical way to teach a low-cost arm a new task, and the published recipe asks for roughly **50 demonstrations per task** (ACT, Zhao et al. 2023; the LeRobot SO-101 tutorial recommends the same order). On real hardware that means a person physically driving a leader arm for **2-3 hours — per task**. For a \$150 arm, the human time dominates the total cost of ownership by a wide margin. Every new task, every changed object, every moved camera pays it again.

What this project does. It moves the bulk of the data cost into simulation, where episodes are free, labelled with ground truth, and reproducible from a seed, and spends human time only on the handful of episodes that anchor the policy to physical reality.

Concretely: a digital twin of the real rig auto-collects 50 demonstrations with a scripted expert, ACT is pretrained on them on an AMD Radeon GPU, and the policy is then fine-tuned on a mixture containing **10 human teleoperation episodes — about 35 minutes of a person's time**. The result runs on the physical SO-101 at **85% success (17/20)**.

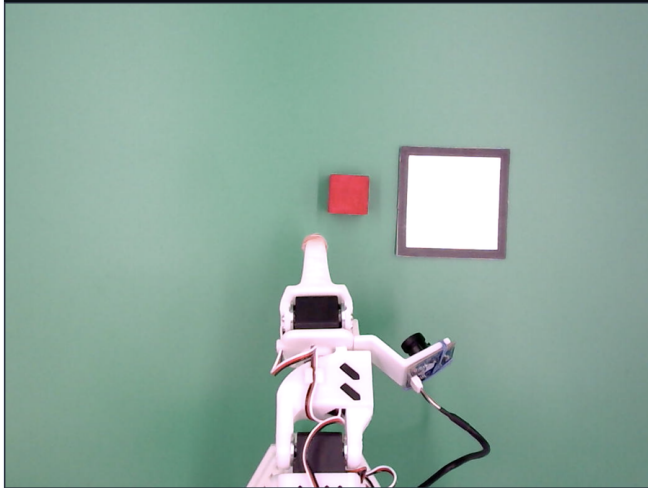
Task. Pick a 4 cm cube off a table and place it on a 10 cm white target sheet. The cube's initial position is randomised over a verified reachable annulus (17-26 cm from the base). Success means the cube comes to rest inside the sheet footprint at table height, judged **after the gripper releases** — an important detail, see §6.1.

Hardware. SO-101: a ~\$150 3D-printed arm with 5 revolute joints plus a single-actuated jaw, driven by Feitech STS3215 servos. Two USB cameras: one overhead, one wrist-mounted.



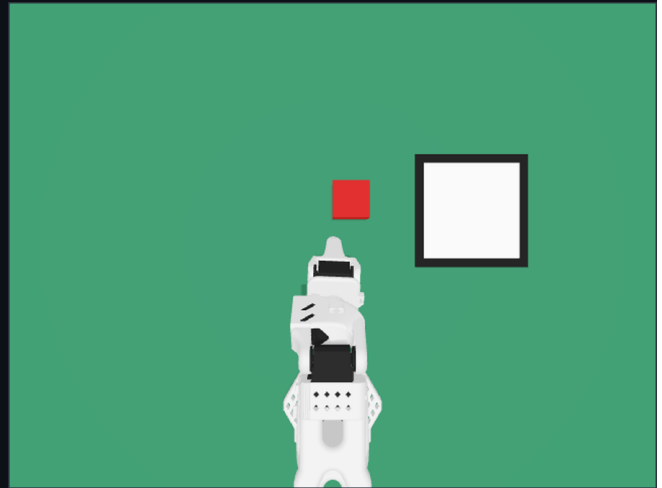
Real SO-101

overhead USB camera, 58 cm



Genesis digital twin

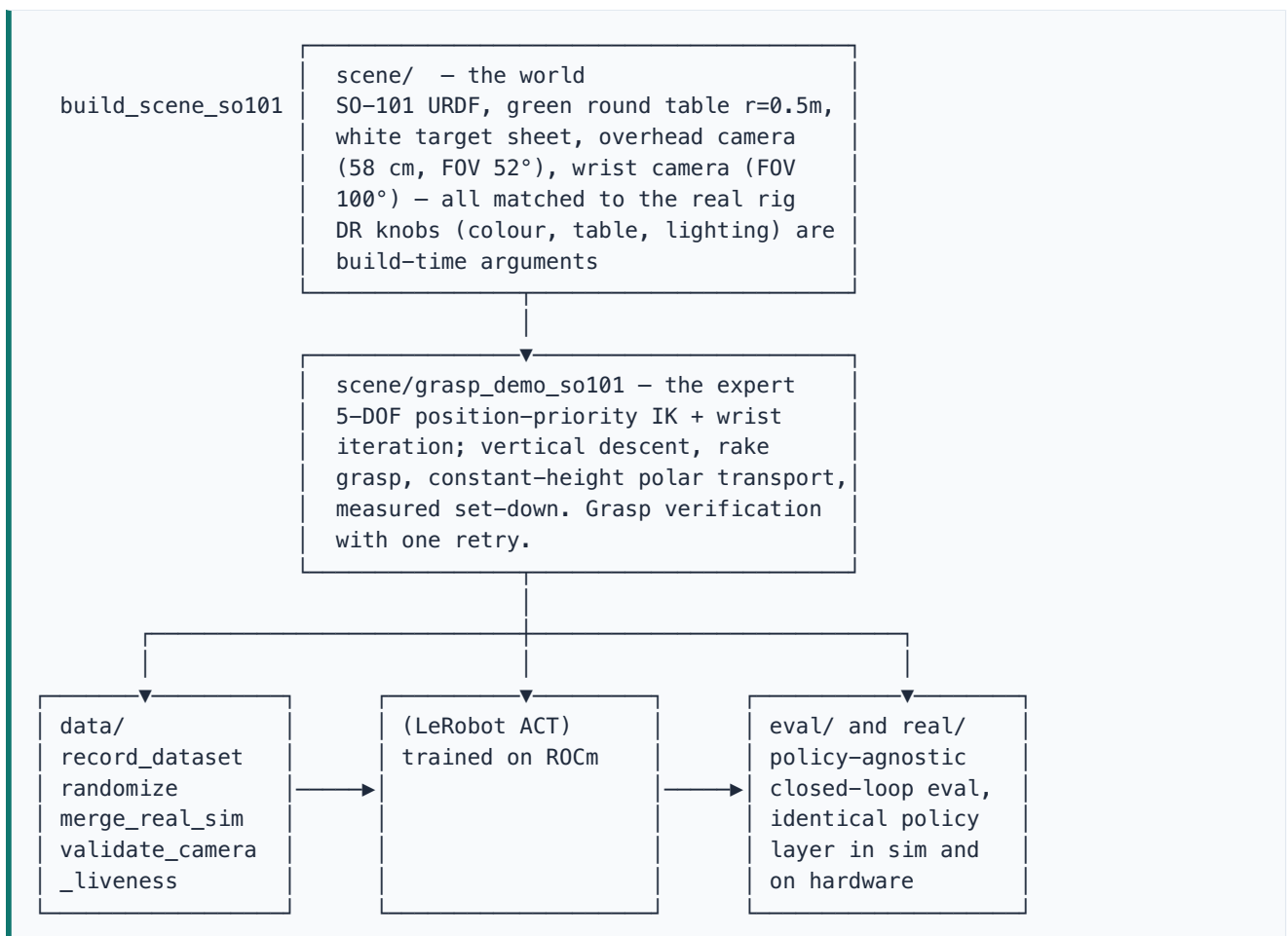
same pose, same FOV, built to measurement



The twin is measured from the rig, not eyeballed: table radius, camera height and field of view, target-sheet size and position.

2. System architecture

Three layers, each independently runnable, so a failure can always be localised.



2.1 The 5-DOF IK problem

This is the single largest engineering difference from the Franka reference demo and worth describing, because it is what porting to a cheap arm actually costs.

A 7-DOF arm like the Franka is redundant: you request a full 6-DOF pose (position + orientation) and a solution almost always exists, with a null space left over. The reference demo's IK is therefore one line —

`inverse_kinematics(link=hand, pos=pos, quat=quat)` — with no error checking, because there is nothing to check.

The SO-101 has **5 arm joints**. Six constraints on five degrees of freedom has no exact solution, and asking the solver for one yields a least-squares compromise that satisfies neither: in this arm's geometry it collapses into a horizontal "crane" posture, useless for a top-down pick. The strategy adopted instead is a **hand-rolled hierarchical IK**:

1. Solve **position only** over all 5 joints, seeded from the home posture. Orientation is whatever falls out.
2. Evaluate the candidate's tool tilt by **forward kinematics** — a predicate on a hypothetical configuration, never touching simulator state.
3. If the tool leans more than 5° from vertical, bend the wrist by exactly the excess and **re-solve position over pan/lift/elbow only**, with the wrist locked.
4. If tilt did not improve, the wrist was bent the wrong way — flip the sign and retry. Iterate up to five times.

Position is the hard constraint (solved exactly by the three large joints); orientation is a soft secondary objective approached iteratively by the wrist. Two further consequences of five DOF shaped the motion primitives:

- **The reachable set is an annulus, not a disk.** A straight xy chord between two reachable points can pass within ~ 0.15 m of the base, where no near-vertical solution exists, so mid-transport waypoints droop and shake the cube loose. Transport therefore interpolates in **polar coordinates** (radius and bearing separately) around the base.
- **Joint-space interpolation bows the fingertip path inward**, enough to knock the cube away during descent. Vertical approach and retreat are subdivided in **Cartesian** space with IK re-solved per waypoint.

The IK is always seeded from the home posture rather than the current configuration. This makes it **deterministic**: the same Cartesian target always yields the same joint solution. For an imitation-learning dataset this matters — a history-dependent seed would label identical camera images with different joint commands, forcing ACT to fit a multi-modal action distribution that carries no task information.

2.2 The gripper is a rake, not a pincer

The SO-101 jaw is single-actuated and swings in the arm's sagittal plane. Measured in-sim from the URDF meshes: fully open, the moving jaw tip sits ~ 9.5 cm beyond and ~ 6 cm above the fixed fingertip, and closing **rakes down and back toward the base**. It scoops; it does not pinch. The expert therefore places the *fixed* fingertip 2.8 cm short of the cube centre **along the base** → **cube bearing** (a bearing-dependent offset, not a fixed xy one) so that closing sweeps the cube against the fixed finger.

Because that grasp is marginal, three behaviours were added that the Franka expert does not need, and they turned out to matter on hardware (§6.3):

- **grasp verification with retry** — lift, check the cube's height, and on failure reopen, re-read where the failed rake pushed the cube, and re-approach;
- **self-calibrating placement** — the cube's offset from the fingertip depends on how the rake pinched it, so read its actual position after transport and correct horizontally;
- **set down, then release** — compute the descent from the held cube's measured height so it touches down rather than falls, and release only partially (opening fully flicks it).

2.3 Simulation physics findings

Tuning contact-rich manipulation in Genesis produced four results worth recording:

Finding	Effect
Collision-mesh convexification destroys the gripper's concave shape	Grasping becomes impossible; <code>convexify=False</code> is mandatory
<code>substeps=2</code> gives contact chaos	<code>substeps=4</code> took grasp success from intermittent to 9/9
Contact friction composes as <code>max()</code> , not a product	Friction DR must be applied to cube, robot and table at the same ratio
<code>noslip_iterations</code> 5 → 10	In-transport drift 1.3 mm → 0.2 mm (measured)

2.4 Sim-to-real bridge

The real LeRobot driver reports arm joints in **degrees** and the gripper in `RANGE_0_100`, while the simulator and datasets use **radians**. A `JointBridge` converts in both directions; a probe mode (torque off, printing both conventions side by side) confirmed the conventions otherwise match within 2-4°, with **no sign flips or offsets needed**.

The real-robot runner shares the policy layer with the sim evaluator, so a checkpoint is driven by identical code in both places. It adds what hardware demands: a per-step joint delta clamp, ease-to-home before torque release on **any** exit path (a raised arm otherwise drops under gravity when an episode aborts), and its own camera transport (§6.1).

3. Datasets

All are LeRobot v3.0 format, 30 fps, `action` = 6-D commanded joint positions, `observation.state` = 6-D measured joint positions, plus 640×480 overhead and wrist RGB.

Dataset	Episodes	Frames	Contents
<code>so101_cube_red</code>	50	~44k	Baseline: red cube, placement randomised only
<code>so101_cube_colors</code>	50	~44k	Cube colour randomised over 25 colours
<code>so101_cube_dr</code>	50	44,513	Full DR: colour + table + lighting + friction + mass
<code>so101_real_teleop2</code>	10	7,268	Real teleoperation via leader arm
<code>so101_real_conv2</code>	10	7,268	The above converted to simulator units
<code>so101_mixed2</code>	80	66,317	Real ×3 (30) + sim 50 → 33.1% real

Two properties are load-bearing for the experiments:

- **The three sim datasets share one seed series**, so cube placements are identical across them and the only difference is the DR content. Any success-rate difference is therefore attributable to DR alone, not to luck in placement. This kind of controlled comparison is only possible in simulation.
- Collection is fully automatic: 50 episodes in ~2 hours with **no human in the loop**, at 94-98% expert success, and ~14% of episodes contain a retry (recovery behaviour that later proved useful on hardware).

Real episodes are oversampled ×3 in the mixture: at 1× the 7,268 real frames would be 14% of each batch and get washed out by the 44,513 sim frames.

4. How the AMD Radeon GPU is used

Stack. `torch 2.9.1+rocm7.2.1.gitff65f5bc` on the Radeon Cloud host. ACT: action chunk 100, hidden dim 512, two ResNet-18 vision towers, VAE latent; batch 8, lr 1e-5.

Training throughput. Every run in this submission was trained on the Radeon GPU.

Run	Data	Steps	Wall clock	Rate	Final loss
① red only	<code>so101_cube_red</code>	100k	~11 h	2.55 step/s	0.028
② colour DR	<code>so101_cube_colors</code>	100k	~11 h	2.55 step/s	0.028
③ full DR	<code>so101_cube_dr</code>	100k	~11 h	2.55 step/s	0.030
(b) co-train FT	<code>so101_mixed2</code> from ③@60k	20k	2 h 11 m	2.56 step/s	0.064
(a) real-only control	<code>so101_real_conv2</code>	20k	2 h 12 m	2.54 step/s	0.103

The GPU is not incidental to the result — it is what made the method testable. Five runs went into this submission: three domain-randomisation ablations that isolate what DR buys, a co-training run, and a control run. Each is an overnight job on this hardware. Without that, the honest version of this project would have been a single training run and a claim; with it, the DR "tax" hypothesis could be tested and refuted, the data ceiling could be located, and the co-training result could be given a control arm.

The GPU was the bottleneck, not the data pipeline, which is the desirable state: `data_s = 0.002 s` against `updt_s = 0.388 s` — **99.5% of wall-clock time was compute**, with VRAM steady at 3.9 GB. Throughput held at 2.54-2.57 step/s across every run and never degraded over 11-hour sessions.

Two jobs share the device gracefully. Running the control experiment alongside the co-training run halved each one's rate (2.55 → 1.56 step/s) and both completed correctly — useful when iterating under a deadline.

Inference. Policy inference during simulated evaluation also runs on the Radeon GPU. On the real robot the policy runs on the host laptop (the arm is physically attached to it), where an ACT chunk replan costs **311 ms** against 3.7 ms for a queue pop — a 3.3-second stutter period on hardware that simulation hides completely, because a simulator waits for computation and a robot does not. This is the clearest argument for GPU-class inference in a real control loop, and it is a limitation of the laptop deployment, not of the method.

ROCm-specific issue and workaround. LeRobot resolves video decoding through `torchcodec`, whose compiled extension is **ABI-incompatible with AMD's custom torch build** and fails at import. All training and evaluation therefore pin `--dataset.video_backend=pyav`, and the Dockerfile never installs `torchcodec`. This is reported upstream (§8).

The whole pipeline runs on the Radeon host, not just training — data collection, training and closed-loop simulated evaluation all execute there, verified on the hackathon Radeon Cloud host (AMD EPYC 9334, Genesis 1.2.3, no display attached). Getting there surfaced two findings about Genesis on ROCm that are worth recording, because neither is documented:

- **`torch.cuda.is_available()` is True on ROCm and it means nothing to Genesis.** ROCm reuses torch's CUDA API, so the obvious check passes — but Genesis computes through Taichi, which has no HIP path, and `gs.cuda` is rejected with "Torch device 'cuda' not available". The fix is to request the generic `gs.gpu` and let Genesis resolve the device, which on the Radeon host becomes `gs.amdgpu`. Policy inference is the opposite case: there `cuda` is correct on ROCm. The two are therefore resolved by separate helpers in `build_scene_so101.py`, and hardcoding either one (the code was written on Apple silicon, so both defaulted to Metal/MPS) makes every script fail on Linux.

- **Offscreen rendering needs EGL.** pyrender's default pyglet path requires a display and fails headless with `IndexError: list index out of range` from `display.get_default_screen()`, so `PYOPENGL_PLATFORM=egl` is set before importing Genesis.

With those two in place, `run_pipeline.sh --quick` completes the full loop — dataset → training → closed-loop evaluation — **on the Radeon GPU in about 3 minutes**: simulation stepping and camera rasterisation on `gs.amdgpu`, ACT training and inference on ROCm. `submission/src/smoke_test_rocm.py` asserts the prerequisites in four checks and is the first thing the reproduction guide asks an evaluator to run.

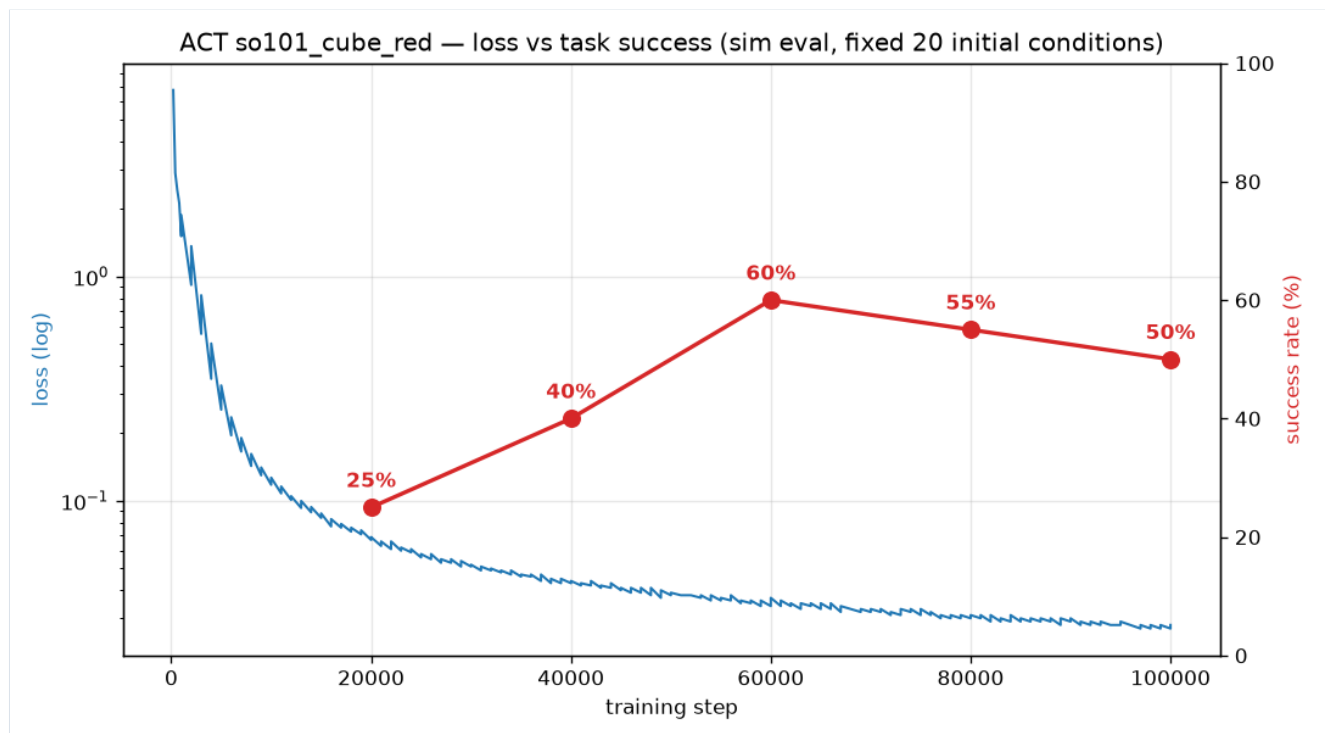
What has not fully transferred, stated plainly: the many-episode collection loop. Genesis' contact solver behaves differently on the ROCm backend — the scripted expert's rake grasp flicks the cube instead of sweeping it in, and the cube slides off the 0.5 m table (`substeps=4` stabilised this on Metal at 9/9 grasps, but is not sufficient there). A single episode is fine: `grasp_demo_so101.py` succeeds on ROCm with 0.3 mm placement error, so the scene and the 5-DOF IK are sound and it is the repeated contact-rich rollouts that need tuning. The submitted datasets were therefore collected on Apple silicon and published on Hugging Face, and the reproduction path downloads them. This is a known limitation with a concrete next step (substep and solver-iteration sweep on the ROCm backend), not a silent gap.

5. Experiments and results

5.1 Data ceiling: success saturates before the loss does

Fixed-seed evaluation of policy ① at five checkpoints — identical 20 placements each, so the comparison is exact.

Step	20k	40k	60k	80k	100k
Success	25%	40%	60%	55%	50%



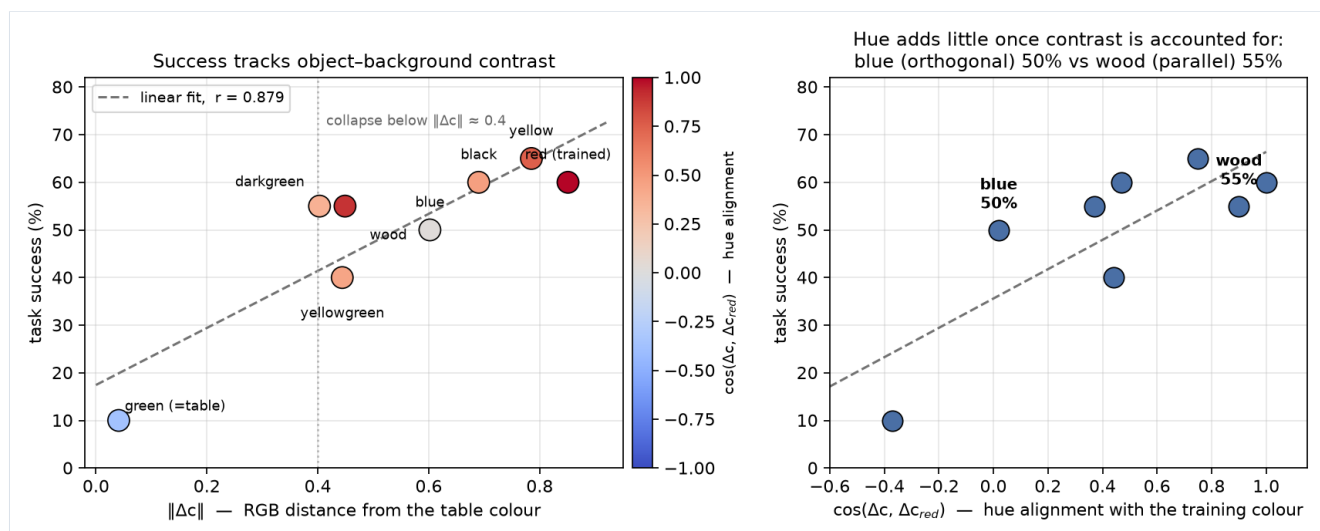
Success peaks at 60k and **declines** while the training loss is still falling monotonically to 0.028. The 50-episode dataset carries about 60% worth of task information for this architecture; further optimisation fits the data, not the

task. The practical reading: when a policy plateaus, more steps are the wrong lever — more data, or better-covering data, is the right one.

5.2 Colour generalisation: contrast, not hue

Policy ① was trained on **red cubes only**, then evaluated zero-shot on seven unseen colours. **20 episodes per colour on identical seeds**, so every colour sees the same 20 placements. Table colour is green (0.22, 0.60, 0.43).

Colour	RGB	$\ \Delta c\ $	cos vs red direction	Success (n=20)
red (trained)	(0.85, 0.12, 0.12)	0.851	1.00	60%
yellow	(0.90, 0.70, 0.05)	0.785	0.75	65%
black	(0.05, 0.05, 0.05)	0.690	0.47	60%
blue	(0.12, 0.18, 0.85)	0.602	0.02	50%
wood	(0.65, 0.50, 0.35)	0.449	0.90	55%
yellowgreen	(0.50, 0.80, 0.15)	0.444	0.44	40%
darkgreen	(0.08, 0.30, 0.20)	0.403	0.37	55%
green (= table)	(0.20, 0.62, 0.40)	0.041	-0.37	10%



The hypothesis going in was that held-out colours would fail. It was wrong. A policy that has never seen a blue cube handles blue at 50%, and one that has never seen black handles black at 60% — the same as the colour it was trained on. What predicts success is $\|\Delta c\|$, the **RGB-space distance between object and background**: correlation **0.879** across the eight colours, and a collapse to 10% only where the cube matches the table.

Hue contributes little, but "nothing" would be an overstatement. The cleanest evidence is the pair that separates the two axes: blue is *orthogonal* to the training colour (cos 0.02) and wood is nearly *parallel* (cos 0.90), yet they score 50% and 55% — indistinguishable at this sample size despite opposite hue alignment. Hue alignment does correlate with success on its own ($r = 0.796$), but that is collinearity: in this palette, colours far from the table also tend to lie along the red direction. Regress $\|\Delta c\|$ out first and hue's marginal correlation with the residual is **0.335** on eight points, i.e. small and not separable from noise. Contrast is doing the work; hue is at most a second-order term.

Human perceptual luminance does not explain this — wood differs from the table by 0.014 in luminance and "should be invisible", yet scores 50%. The mechanism is that ResNet's first convolution holds independent weights per RGB channel, so its response difference is proportional to $w \cdot \Delta c$; no projection onto a single

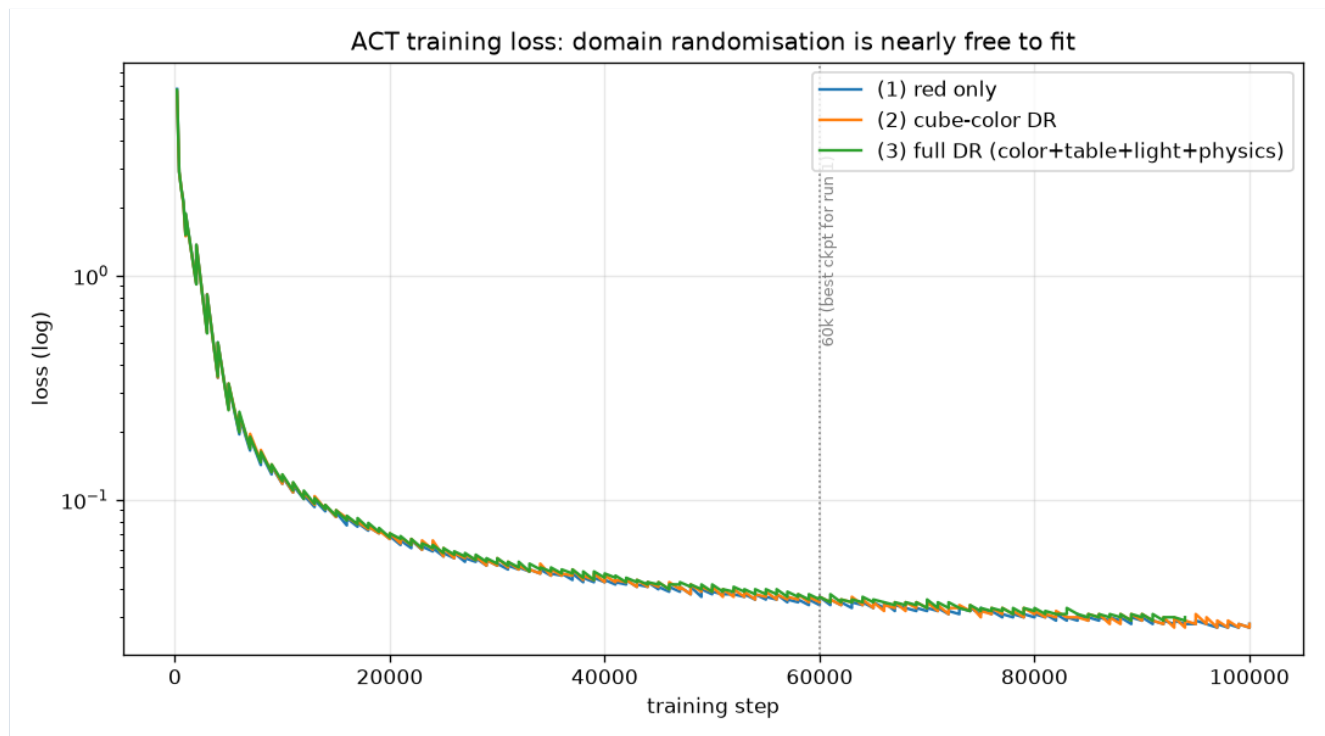
luminance axis ever happens. With stride 32, a 4 cm cube also occupies roughly **one token out of ~300**, which is why the failures at low contrast are not "searching in the wrong place" — in 6 of 9 failing episodes the arm still reached the cube's vicinity, and the collapse is a prior collapse toward the training set's average trajectory.

Practical consequence, and it inverts the usual advice: before collecting data, measure the object-to-background RGB difference. The useful instruction is not "randomise colours" but "**keep $\|\Delta c\|$ above ~0.4**".

Caveat: 20 episodes per colour, so each point still carries roughly ± 11 percentage points of binomial noise — the darkgreen (55%) versus yellowgreen (40%) inversion against their nearly equal $\|\Delta c\|$ sits inside that. An earlier 10-episode round gave $r = 0.956$; the extra episodes moved it to 0.879, which is the number to trust. Single task, single flat background (a textured one cannot be summarised by a scalar $\|\Delta c\|$), single architecture, simulation only.

5.3 Domain randomisation is nearly free

A second hypothesis — that DR would cost representational capacity and slow fitting — was also refuted. Final losses at 100k: **0.028 (no DR) / 0.028 (colour DR) / 0.030 (full DR)**. Learning curves are near-identical — the three traces below overlap almost everywhere. DR's robustness comes at essentially no fitting cost, which is a useful thing to know before deciding how much of it to enable.



5.4 Real-robot zero-shot transfer

Policies ① and ③ were run directly on hardware. **Visual transfer works** — from real camera images the arm reaches the real cube — but the grasp does not complete. Domain randomisation improved the visual side without fixing the grasp, so the residual gap is not appearance. Three gap components were measured independently:

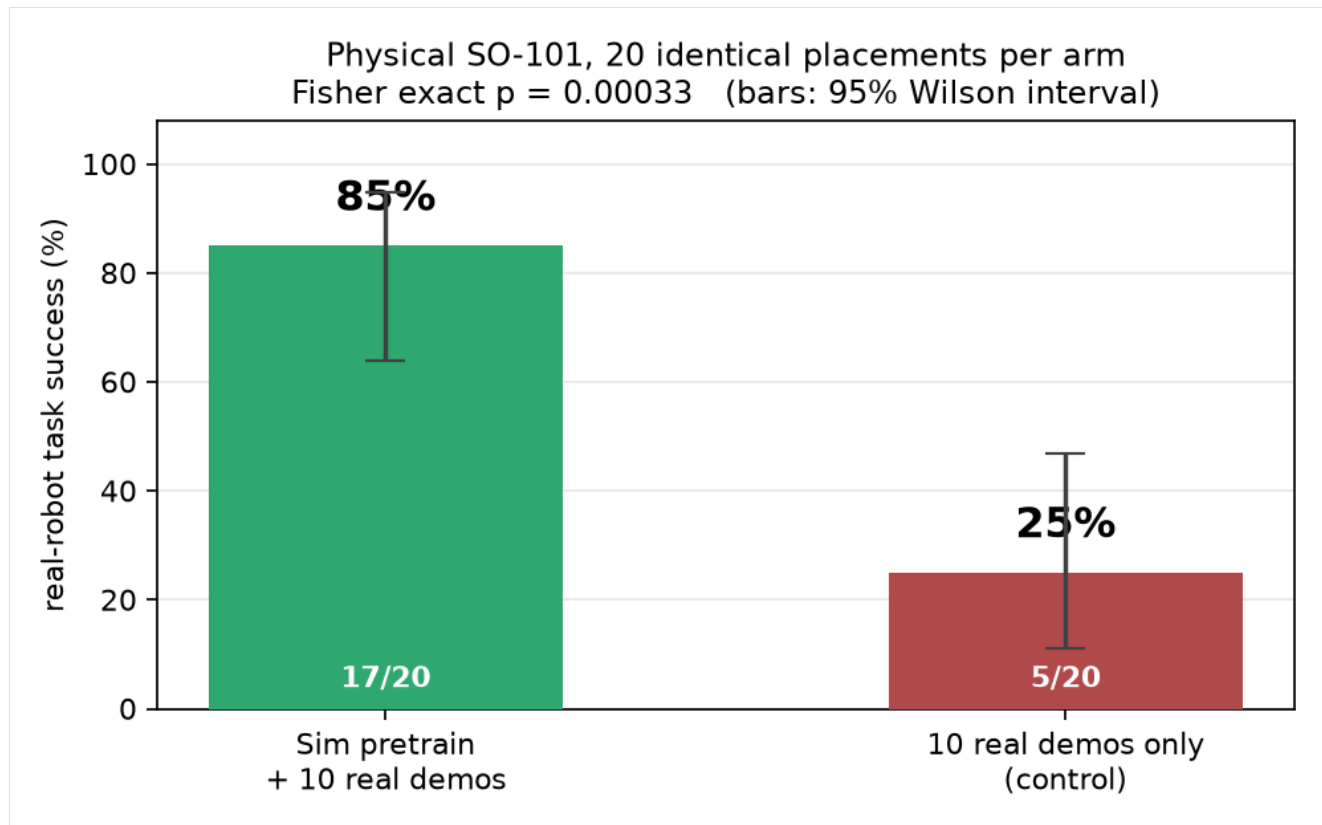
1. **Control gain:** simulated KP = 120 against the servo's P = 16, so tracking is sluggish for the same commanded angle.
2. **Wrist camera mass:** a camera on the wrist applies a static torque absent from the model.
3. **Inference latency:** the 311 ms chunk replan described in §4.

A conclusion originally drawn here — "the residual gap is dynamics" — has been retracted. See §6.1: that evaluation ran with a broken wrist camera and cannot support the claim.

5.5 Co-training with 10 real episodes

	Training	Data	Epochs	Real success
(b)	③ full-DR 60k → fine-tune	real 30 (10×3) + sim 50, 66,317 frames	2.41	85% (17/20)
(a)	ACT from scratch (control)	real 10 only, 7,268 frames	22.0	25% (5/20)

Steps, batch size, learning rate and image augmentation are identical (20k / 8 / 1e-5 / enabled), so the only difference is the presence of sim pretraining and sim data. Both were evaluated on the **same 10 placements**.



- (b) places the cube **1-2 cm** from the sheet centre (12-23% of the sheet width).
- The two failures were the same mode: the rake misses, the cube shifts, the policy re-approaches but cannot grasp.
- **Fisher exact test: $p = 0.00033$ two-tailed.** 95% confidence intervals do not overlap (64-95% against 11-47%), so the effect is established rather than suggested: for this task and rig, **the simulated pretraining is what makes 10 real demonstrations enough**. An earlier 10-episode-per-arm round gave 80% vs 30% at $p = 0.070$ — the same direction but underpowered, which is why the evaluation was extended to 20.

Two observations are solid regardless of that test. First, **10 real episodes alone already get 30%**, which is itself encouraging for anyone teaching a cheap arm: the conventional 50 is not a cliff. Second, at 22 epochs over 10 episodes the control run is deep into overfitting territory while the co-trained run sits at 2.41 epochs — the sim data is doing regularisation work as well as providing coverage.

5.6 Sim-designed recovery behaviour fires on hardware

In several successful real episodes the policy **missed the grasp, re-approached, and succeeded** — the verification-and-retry behaviour built into the *simulated* scripted expert (§2.2), present in ~14% of training episodes, generalising to real-world failures it never saw. This is the cleanest sim-to-real result in the project: a recovery strategy designed in simulation working against physical failure modes.

6. Innovations and key technical contributions

6.1 A failure mode that every standard check passes

This was the most consequential engineering finding, and it invalidated a full day of work.

The wrist camera entered a state where `cap.read()` returned `ok=True` with a stale buffer — frames kept arriving, pixels never changed. All ten initial teleoperation episodes were recorded with a **frozen wrist view**, and one real evaluation ran with the wrist frozen for **43.8 of 45 seconds (97%)**.

Nothing caught it:

- shape, dtype, unit ranges, fps, file sizes and playability were all **valid**;
- the overhead camera was fine, so spot-checking the "action" view showed nothing;
- **the training loss was identical to a clean run** (0.065 vs 0.064 at 20k; weight drift from the pretrained base 2.305% vs 2.315%). A constant image is trivially predictable, so if anything it *helped* the loss. **Loss is not a data-quality signal**.

Only one check detects it: **pixel change over time**. Two tools now do this and ship with the submission — `data/validate_camera_liveness.py` (per-episode, per-camera longest static run, plus gripper-cycle and tracking-error checks) and `eval/audit_rollout_videos.py` (labels past rollout videos `CLEAN` or `CONFOUNDED`).

Root cause, and it was not the hardware. The trigger was our own **resolution request**: asking this device for 320×240 selects its 120 fps mode, which stalls every ~9 s; 640×480 stalls every ~15 s; the **native 1920×1080@30 mode soaked for 150 s with zero stalls**. USB port changes and hub avoidance made no difference. The decisive clue came from the operator's observation that Photo Booth never froze — which pointed at how *we* were opening the device rather than at the device itself. Both the recorder and the real-robot runner now open the camera with **no property requests**, then centre-crop and resize in software.

There is an uncomfortable irony worth recording. LeRobot's camera layer *has* a staleness check that aborts on a >500 ms stale frame. It fired spuriously against these devices — the overhead camera reports 5 fps while delivering ~15, the wrist camera's fps report flaps with the negotiated resolution — so we replaced the transport with one that "never raises". In suppressing the false positives we discarded the one true positive that mattered. The fix is not to remove the check but to make it correct: a **liveness guard** now refuses to start if a camera is open but static, reopens the device after 2 s of no change, and aborts after 10 s.

6.2 Other contributions

- **A 5-DOF IK strategy for cheap arms** (§2.1): position-priority solve with FK-verified wrist iteration, plus polar transport to avoid the unreachable annulus centre. The reference demo's one-line 6-DOF IK does not port to a 5-DOF arm, and this is what replaces it.
 - **Controlled DR ablation** (§5.2, §5.3): identical seed series across three datasets makes DR the only variable, which is what allowed two plausible hypotheses to be refuted rather than assumed.
 - **The contrast finding** (§5.2) as an actionable rule: measure $\|\Delta c\|$ before collecting.
 - **A reproducible-on-AMD-alone path**: an evaluator without an SO-101 can run collection, training and simulated evaluation end to end in the provided container.
-

7. Deliverables

Deliverable	Where
Technical report	<code>docs/Technical_Report.md</code> / <code>.pdf</code>
Demonstration video	<code>docs/demo_video.mp4</code>
Source code	<code>submission/src/</code> (scene, data, train, eval, real)
Docker environment	<code>Dockerfile</code> (ROCm base, EGL headless, pyav pinned)
Reproduction guide	<code>submission/README.md</code>
Trained model	https://huggingface.co/omiya239532/so101_act_cotrain
Datasets (5, public)	<code>omiya239532/so101_cube_red</code> , <code>so101_cube_colors</code> , <code>so101_cube_dr</code> , <code>so101_real_teleop2</code> , <code>so101_mixed2</code>
Validation tooling	<code>data/validate_camera_liveness.py</code> , <code>eval/audit_rollout_videos.py</code> , <code>src/smoke_test_rocm.py</code>

8. Upstream contribution

Reported to **LeRobot**: UVC cameras that keep returning `ok=True` with an unchanging buffer are recorded as valid datasets, because no layer verifies pixel change over time. The report includes the resolution-mode trigger, quantified soak data (stall interval per mode), the identical-loss evidence that training cannot detect it, and the liveness-guard implementation used here. Evidence in `docs/upstream_contribution_evidence.*`.

Also documented for the ROCm ecosystem: the `torchcodec` ABI incompatibility with AMD's custom torch build, with the `pyav` workaround (§4).

9. Limitations

Stated plainly, because several of these were discovered the hard way:

- The co-training advantage is established for **this task and this rig** (n = 20 per arm, p = 0.00033), not shown to generalise across tasks, objects or camera setups.
- The contrast finding rests on 10-20 episodes per colour on a **single flat background**; a textured background cannot be summarised by a scalar $\| \Delta c \|$.
- One task, one object, one camera rig. No claim of cross-task generality.
- The real-robot control loop runs on a laptop, where the 311 ms chunk replan forces smoothing and half-speed playback. GPU-class inference at the robot would remove this.
- **§5.4's original conclusion was retracted** after the camera fault was found. The independently measured gap components stand; the synthesis built on that evaluation does not.

10. Team

Solo submission — **omiya** (Fusic Co., Ltd.).

All work in this repository: the SO-101 port of the scene and scripted expert, the 5-DOF IK strategy, dataset collection and domain-randomisation tooling, all five training runs on the Radeon host, the simulated and real evaluation harnesses, real-robot bring-up (calibration, unit bridge, camera alignment, teleoperation recording), the five experiments above, and the diagnosis and repair of the camera fault in §6.1.

The scene/expert/recording structure follows the Track 3 reference demo [wangxunx/franka_fruit_pick_demo](#) (Apache-2.0), which this project ports from a 7-DOF Franka in simulation to a 5-DOF low-cost arm on real hardware.